

It has been standard in NLP for the last 15 or so years to represent words using vectors. There are many ways to do this, but the most straightforward way, which we will be using here, is to represent each word form by one vector. The dimensionality of these vectors varies, but 300 dimensions (as we will be using) is a common choice. We are using multilingual word vectors from the ConceptNet Number-batch project, which provides 300-dimensional word vectors for 79 different languages. You will implement a system to search through a database of news text. The system should be able to accept a search query in one language and return hits in another language. The file format is simple: the first line contains two space-separated numbers, the vocabulary size (varies by language) and the dimensionality (in the data above, this is always 300). All following lines contain a word, followed by the vector itself as a list of 300 floats. Your task is to write two classes in the file `text_search.py`: `WordVectors` and `TextSearch`. The class `WordVectors` should have the following methods:

- A constructor that takes a filename containing an embeddings file of the format we use (see above).
- `make_sentence_vector(words: list[str]) -> np.ndarray` that computes the mean vector of all the word types in the given sentence.

Only words that occur in the vocabulary of the word vectors should be included, and each word should be counted only once even if it occurs multiple times. The word vectors only contain lower-case words, so all words in the input should be lower-cased. The returned array should be a vector of the same dimensionality as the word vectors (in this case, 300, but your code should be general enough to handle other cases as well). If none of the words has a word vector associated with it, raise a `ValueError` exception. Use the data type `numpy.float32` when creating the NumPy vectors, in order to conserve memory (see the NumPy introduction). The class `TextSearch` should have the following methods:

- A constructor that takes a list of filenames with word vectors. The name of the file is of the form `de.vec.gz` (for German, with language code `de`). This is important, so that you can keep track of which language each file's word vectors refers to, which allows you to support searches in many different languages.
- `index_text(filename: str, language_code: str, min_words: int= 1, max_words: int = None) -> None` which takes a filename of a gzip-compressed text with one tokenized sentence per line, reads it and computes and stores sentence vectors for each sentence. The exception is sentences with no words in the vocabulary of the given language's word vectors, or sentences that are shorter or longer than the min words and max words arguments specify. If one or both of the limits is `None`, that particular limit(s) does not apply. Such sentences should be ignored and not indexed. Use the `make_sentence_vector()` method to compute the sentence vector.
- `search(query: list[str], language_code: str, n_matches: int=1) -> list[tuple[float, str, str]]` that returns then matches (with a default value of 1) sentences most similar to the query, among all of the texts that have been loaded, with the most similar first. Each tuple in the returned list should contain the following three items: the cosine similarity between the query and the sentence, the file name of the text file (the same that was passed to `index_text()`), the sentence itself as a string, exactly as found in the input text file. Note that this method should not read any files, but rely on pre-computed data stored in the `WordVector` object.

Next task: implement inverted indexing to create word vectors. You can use the multi-parallel data from the European parliament. The first part of the file name contains the language code, just as in the word vector files we used in the first part. Produce the files `de.vec.gz`, `en.vec.gz`, etc. of the same format as ConceptNet Numberbatch.